

UNIVERSIDADE FEDERAL DE GOIÁS

Instituto de Informática

Software Concorrente e Distribuído

CityPulse

Plataforma de Monitoramento de Cidade Inteligente

Documentação Técnica

Integrantes do Grupo

Mário César Paulino Ferro *(Sensores Simulados — Node.js)*

Osmário Santos Lima Filho *(Coletor de Zona — Java)*

Allan Bispo dos Santos *(Gateway de API — NestJS)*

Douglas Eduardo Rocha de Sousa *(Dashboard & Worker de Manutenção — Next.js)*

Sumário

1. Visão Geral do Projeto
2. Tema e Contexto Acadêmico
3. Arquitetura do Sistema
 - 3.1 Diagrama de Componentes
 - 3.2 Particionamento Geográfico
 - 3.3 Replicação Primário-Réplica
4. Linguagens e Tecnologias
5. Protocolos de Comunicação
 - 5.1 MQTT — Publish/Subscribe
 - 5.2 gRPC — Chamada Remota Síncrona
 - 5.3 WebSocket (Socket.IO) — Push em Tempo Real
 - 5.4 AMQP (RabbitMQ) — Mensageria Assíncrona
 - 5.5 REST (HTTP) — Interface Cliente-Servidor
6. Concorrência
 - 6.1 Concorrência no Coletor de Zona (Java)
 - 6.2 Concorrência nos Serviços TypeScript
7. Serviços em Detalhe
 - 7.1 Sensores Simulados
 - 7.2 Coletor de Zona
 - 7.3 Gateway de API
 - 7.4 Dashboard
 - 7.5 Worker de Manutenção
8. Armazenamento de Dados
9. Separação de Instâncias e Deploy
 - 9.1 Ambiente de Desenvolvimento (Docker Compose)
 - 9.2 Ambiente de Produção (AWS EC2)
10. Fluxos de Dados Principais
11. Considerações Finais

1. Visão Geral do Projeto

CityPulse é uma plataforma de monitoramento de cidade inteligente (Smart City) desenvolvida como trabalho final da disciplina Software Concorrente e Distribuído. O sistema coleta, processa, agrega e exibe em tempo real dados de sensores urbanos distribuídos por quatro zonas geográficas de uma cidade fictícia: Centro, Norte, Sul e Leste.

A plataforma é intencionalmente projetada para demonstrar, na prática, os principais conceitos estudados na disciplina: concorrência explícita com múltiplas threads e estruturas de dados thread-safe, distribuição de carga por particionamento, replicação com failover automático, interações síncronas e assíncronas entre componentes heterogêneos, e deploy em infraestrutura de nuvem com instâncias físicas separadas.

Objetivos do Sistema

- Simular sensores IoT de temperatura, qualidade do ar, vagas de estacionamento e consumo de energia.
- Coletar e processar leituras em tempo real por zona geográfica, com múltiplas threads concorrentes.
- Detectar violações de limites configuráveis e publicar alertas automaticamente.
- Replicar o estado de cada zona para uma instância secundária com promoção automática em falhas.
- Expor dados via API REST, WebSocket e gRPC para consumo pelo dashboard e outros clientes.
- Demonstrar interoperabilidade entre Java (JVM) e TypeScript (Node.js) via gRPC e mensageria.
- Executar em produção em cinco instâncias EC2 separadas na AWS com comunicação via VPC.

2. Tema e Contexto Acadêmico

A motivação para o tema de Smart City vem da crescente necessidade de sistemas de monitoramento urbano em tempo real. Cidades modernas lidam com grandes volumes de dados gerados continuamente por sensores físicos — temperatura ambiente, qualidade do ar, ocupação de estacionamentos, consumo de energia elétrica — e precisam processá-los de forma eficiente, confiável e escalável.

Do ponto de vista acadêmico, esse domínio é ideal para explorar os desafios de sistemas distribuídos porque naturalmente exige: particionamento geográfico dos dados (uma zona por coletor), tolerância a falhas (réplicas), comunicação assíncrona de alta frequência (sensores publicando a cada 5–15 segundos via MQTT), comunicação síncrona bloqueante para operações de escrita consistente (gRPC com confirmação dupla) e processamento concorrente de leituras simultâneas de múltiplos sensores.

Métricas Monitoradas

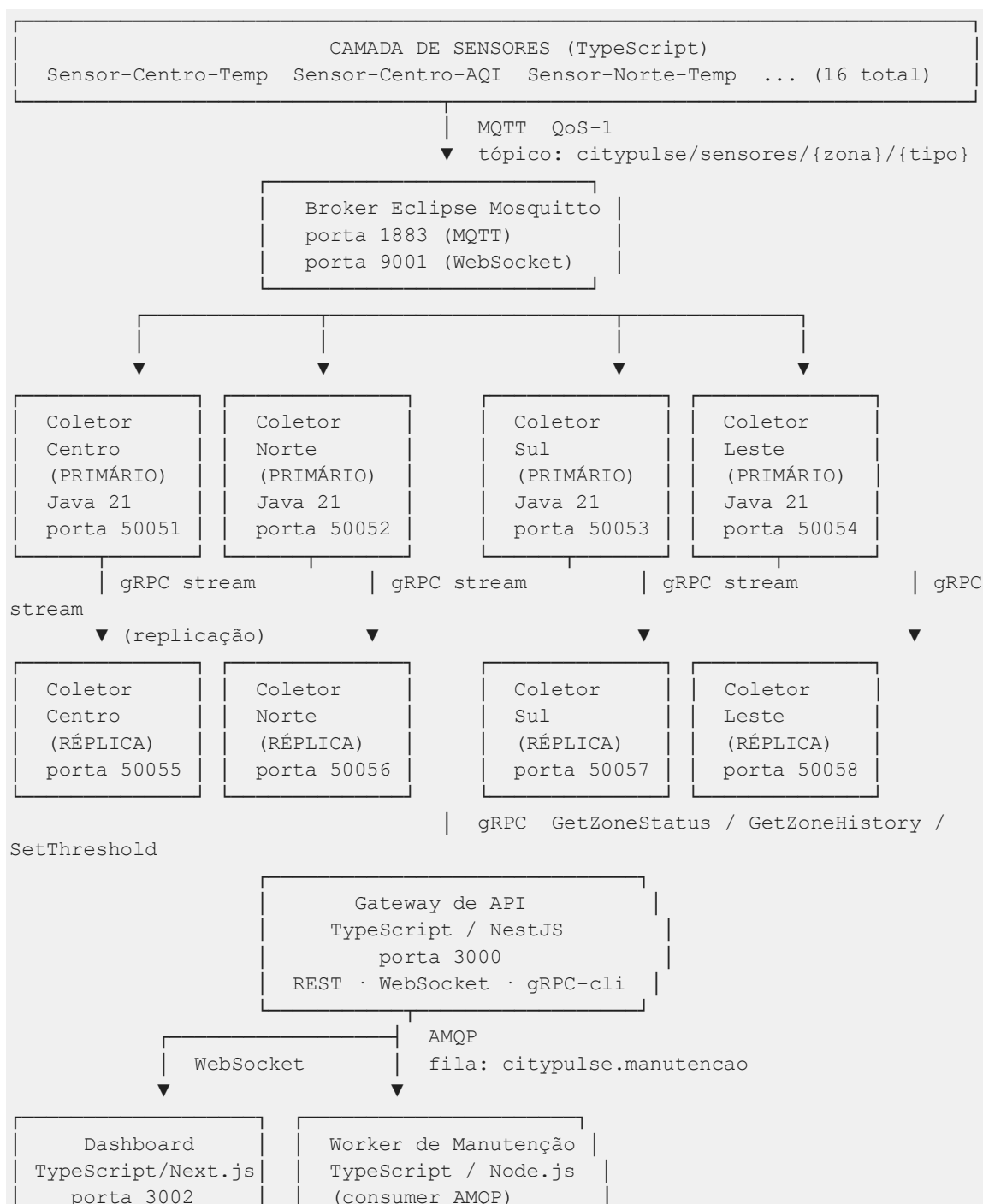
Tipo de Sensor	Unidade	Faixa Normal	Outlier (descartado)
Temperatura	°C	-5 a 45 °C	< -40 ou > 60 °C filtrado
Qualidade do Ar (AQI)	AQI	0 a 500	< 0 ou > 1000 filtrado
Vagas de Estac.	vagas	0 a 100	< 0 ou > 100 filtrado
Consumo de Energia	kWh	variável c/ hora	valores negativos filtrados

Cada tipo de sensor possui comportamento simulado realista: temperatura caminha suavemente com variação máxima de $\pm 1,5$ °C por leitura; qualidade do ar tem leve tendência de piora (bias de +0,45 AQI); vagas de estacionamento simulam entrada e saída de veículos; consumo de energia varia com a hora do dia, simulando picos diurnos (8h–18h) e vales noturnos (22h–6h). Em 5% das leituras, cada sensor pode gerar um valor de outlier intencional para testar o Sanitizador do Coletor de Zona.

3. Arquitetura do Sistema

A arquitetura do CityPulse é baseada em microserviços distribuídos, com cinco tipos de serviço distintos organizados em camadas. Os serviços se comunicam por quatro protocolos diferentes (MQTT, gRPC, WebSocket e AMQP), cada um escolhido conforme o padrão de interação necessário.

3.1 Diagrama de Componentes





3.2 Particionamento Geográfico

A cidade fictícia é dividida em quatro zonas geográficas: Centro, Norte, Sul e Leste. Cada zona é uma partição independente de dados, processada por sua própria instância do Coletor de Zona. Esse modelo de particionamento horizontal distribui tanto a carga de processamento quanto o armazenamento em memória entre quatro processos JVM independentes.

Cada Coletor de Zona subscreve apenas no tópico MQTT correspondente à sua zona (ex.: `citypulse/sensores/centro/+`), processa somente leituras de sensores daquela partição, mantém seu próprio estado em memória (`ConcurrentHashMap`) e expõe sua própria porta gRPC. O Gateway de API conhece o endereço de cada Coletor e roteia as requisições REST do dashboard para a partição correta.

3.3 Replicação Primário-Réplica

Cada partição (zona) possui dois Coletores: um PRIMÁRIO e uma RÉPLICA. O Primário processa todas as leituras de sensores, avalia violações de limites e publica alertas. Ele mantém a Réplica atualizada enviando continuamente o estado por meio de um stream gRPC bidirecional, com heartbeats a cada 2 segundos.

A Réplica recebe as atualizações de estado e as aplica em sua própria cópia do `EstadoZona`. Um watchdog na Réplica verifica a cada 1 segundo se um heartbeat chegou nos últimos 6 segundos. Se o timeout ocorre — indicando queda do Primário — a Réplica promove a si mesma a PRIMÁRIO via `AtomicReference` com `compareAndSet` (operação lock-free), passando a aceitar e processar leituras MQTT e a publicar alertas.

Quando um `SetThreshold` (atualização de limite) é recebido pelo Gateway, ele é propagado de forma síncrona tanto para o Primário quanto para a Réplica, e o Gateway só retorna `sucesso=true` ao cliente quando ambos confirmam — garantindo consistência forte para essa operação de escrita.

4. Linguagens e Tecnologias

O projeto usa deliberadamente mais de uma linguagem de programação para demonstrar interoperabilidade em sistemas distribuídos. Java é usado no componente de maior exigência de concorrência explícita (Coletor de Zona), enquanto TypeScript é usado em todos os demais serviços, cada um com um framework específico para sua função.

Serviço	Linguagem	Runtime	Frameworks / Bibliotecas	Responsabilidade Principal
Sensores Simulados	TypeScript	Node.js 20	mqtt v5, Express 4	Publicação MQTT periódica
Coletor de Zona	Java 21	JVM (Temurin 21)	gRPC-Java 1.68, Paho MQTT, Jackson	Processamento concorrente e replicação
Gateway de API	TypeScript	Node.js 20	NestJS 10, @grpc/grpc-js, Socket.IO, amqplib	Roteamento e orquestração
Dashboard	TypeScript	Node.js 20	Next.js 14, React 18, socket.io-client	Interface web em tempo real
Worker de Manutenção	TypeScript	Node.js 20	amqplib	Consumer de fila AMQP

Infraestrutura de Suporte

Componente	Tipo	Portas	Função
Eclipse Mosquitto 2.0	Broker MQTT	1883 (MQTT), 9001 (WS)	Mensagens sensores → coletores e alertas
RabbitMQ 3.13	Broker AMQP	5672 (AMQP), 15672 (UI)	Fila de tarefas Gateway → Worker
Docker / Compose	Containerização	—	30 containers na rede bridge citypulse
Protocol Buffers 3	Serialização IDL	—	Contrato gRPC entre Java e TypeScript
Amazon EC2	Infraestrutura Cloud	AWS VPC	5 instâncias t3.micro/small na produção

5. Protocolos de Comunicação

Um dos requisitos explícitos da disciplina é o uso de múltiplos protocolos de comunicação, cada um adequado ao seu padrão de interação. O CityPulse usa quatro protocolos distintos: MQTT, gRPC, WebSocket (Socket.IO) e AMQP (RabbitMQ), além de REST/HTTP como interface para o dashboard.

5.1 MQTT — Publish/Subscribe Assíncrono

MQTT (Message Queuing Telemetry Transport) é um protocolo leve de pub/sub projetado para IoT. É a tecnologia ideal para o fluxo de dados dos sensores: publicação unidirecional de alta frequência, com qualidade de serviço configurável e desacoplamento total entre produtores (sensores) e consumidores (coletores).

- Broker: Eclipse Mosquitto 2.0, porta 1883
- QoS 1 (at-least-once): garante entrega mesmo com reconexões
- Tópico de leituras: `citypulse/sensores/{zona_id}/{tipo}`
 - Payload: { sensor_id, zona_id, tipo, valor, unidade, timestamp }
- Tópico de alertas: `citypulse/alertas/{zona_id}`
 - Payload: { zona_id, tipo, nivel, valor, limite, mensagem, timestamp }
- Apenas o Coletor PRIMÁRIO publica alertas — a Réplica não duplica publicações
- Reconexão automática habilitada (`setAutomaticReconnect=true`) no cliente Java Paho

5.2 gRPC — Chamada Remota Síncrona (HTTP/2 + Protocol Buffers)

gRPC é usado para duas finalidades distintas: comunicação síncrona entre o Gateway e os Coletores (leitura de estado e configuração de limites) e comunicação interna de replicação de estado entre Primário e Réplica. O uso de Protocol Buffers garante esquema forte e serialização eficiente, viabilizando interoperabilidade entre Java e TypeScript.

- Serviço ZoneCollector (Gateway → Coletores):
 - `GetZoneStatus`: lê métricas atuais, papel (PRIMARIO/REPLICA) e timestamp
 - `GetZoneHistory`: lê histórico de DataPoints por tipo de métrica
 - `SetThreshold`: configura limite com confirmação síncrona dupla (Primário + Réplica)
- Serviço ReplicacaoColetor (Primário → Réplica):
 - `replicarEstado`: stream bidirecional com heartbeats a cada 2 s e deltas de estado
 - `aplicarLimite`: chamada unária para propagar `SetThreshold` à Réplica
- Failover automático no Gateway: erro UNAVAILABLE ou DEADLINE_EXCEEDED → redirect para Réplica

5.3 WebSocket (Socket.IO) — Push em Tempo Real

WebSocket é utilizado para entregar alertas e atualizações de status ao Dashboard sem polling. O Gateway mantém um servidor Socket.IO e organiza clientes em rooms por zona, de modo que uma mensagem enviada para o room zona:centro alcança apenas os clientes que monitoram aquela partição.

- Servidor Socket.IO no Gateway, porta 3000 (mesmo endpoint do REST)
- Rooms por zona: zona:centro, zona:norte, zona:sul, zona:leste
- Subscription: cliente emite evento subscribe com { acao: "subscribe", zona_id }
- Eventos emitidos: alerta (payload de alerta MQTT) e status_update
- Singleton socket.io-client no Dashboard — uma única conexão compartilhada por toda a SPA

5.4 AMQP (RabbitMQ) — Mensageria Assíncrona

AMQP via RabbitMQ é usado para comunicação assíncrona e durável entre o Gateway e o Worker de Manutenção. A fila é declarada como durable e as mensagens como persistent, garantindo que tarefas não sejam perdidas em caso de reinicialização do broker.

- Fila: citypulse.manutencao (durable=true, persistent=true)
- Publicador: Gateway publica após SetThreshold bem-sucedido
- Consumer: Worker com prefetch(1) — processa uma tarefa por vez
- Tipos de tarefa: reconciliacao | downsampling | relatorio
- Payload: { tipo, zona_id, agendado_em }
- Retry de conexão: 30 tentativas com intervalo de 2 s no Worker

5.5 REST (HTTP) — Interface Cliente-Servidor

REST é a interface principal entre o Dashboard e o Gateway, usada para consultas síncronas de estado e configuração de limites. CORS está aberto para qualquer origem, adequado ao ambiente de desenvolvimento e demonstração acadêmica.

Método	Endpoint	Descrição
GET	/api/zonas	Lista todas as zonas
GET	/api/zonas/:id/status	Status atual da zona (métricas + papel)
GET	/api/zonas/:id/historico	Histórico de DataPoints da zona
GET	/api/alertas	Últimos 500 alertas (ring buffer)
POST	/api/zonas/:id/limites	Configura limite — dispara SetThreshold gRPC
POST	/sensores/registrar	Registra sensor no serviço de ingestão (porta 3001)

6. Concorrência

A concorrência é um requisito central da disciplina. O CityPulse demonstra dois modelos distintos de concorrência: concorrência explícita baseada em threads (Java, no Coletor de Zona) e concorrência cooperativa baseada em event loop (TypeScript/Node.js, em todos os demais serviços).

6.1 Concorrência no Coletor de Zona (Java 21)

O Coletor de Zona é o componente onde a concorrência é mais elaborada e explicitamente demonstrada. Múltiplas threads operam simultaneamente sobre o estado compartilhado da zona, exigindo estruturas de dados e mecanismos de sincronização adequados.

Estruturas de Dados Thread-Safe (EstadoZona.java)

```
// Acesso concorrente por 4+ threads sem locks explícitos no caminho quente
ConcurrentHashMap<String, Metrica>                metricaAtual
ConcurrentHashMap<String, ConcurrentLinkedDeque<DataPoint>> historico // MAX
5.000 pts/tipo
ConcurrentHashMap<String, Limite>                  limites
```

ConcurrentHashMap oferece operações atômicas de leitura e escrita sem bloqueio global (lock striping). ConcurrentLinkedDeque é uma fila duplamente encadeada lock-free adequada para múltiplos produtores e consumidores. Essas estruturas permitem que as threads de ingestão MQTT escrevam concorrentemente enquanto o servidor gRPC lê e o Agregador calcula médias — tudo sem locks explícitos nos caminhos quentes.

Threads Concorrentes no Coletor

Thread / Pool	Cardinalidade	Mecanismo	Responsabilidade
Pool de Ingestão MQTT	4 threads fixas (ingestao-0 a ingestao-3)	ExecutorService fixed	Processa leituras MQTT em paralelo: Sanitizador → EstadoZona → AvaliadorLimites
Servidor gRPC (Netty)	Gerenciado pelo framework	Netty I/O threads	Atende GetZoneStatus e GetZoneHistory lendo o EstadoZona concorrentemente
Agregador	1 thread (agregador)	ScheduledExecutorService	Calcula médias por minuto a cada 60 s, lendo o histórico concorrentemente
Replicação — Primário	1 thread (replicacao-primario)	ScheduledExecutorService	Envia heartbeats + deltas de estado via stream gRPC a cada 2 s
Watchdog — Réplica	1 thread (replicacao-watchdog)	ScheduledExecutorService	Verifica heartbeat a cada 1 s; promove a Primário após 6 s de silêncio
Servidor gRPC Réplica	Gerenciado pelo framework	Netty I/O threads	Aceita stream de replicação e aplicarLimite do Primário

Promoção Lock-Free (GerenciadorPapel.java)

A promoção de REPLICA para PRIMARIO é uma operação crítica que deve ser atômica e livre de condições de corrida. O GerenciadorPapel usa AtomicReference<Papel> com compareAndSet para garantir que apenas uma thread promova o Coletor, mesmo que watchdog e handler de erro de stream disparem simultaneamente:

```
AtomicReference<Papel> papel = new AtomicReference<>(Papel.REPLICA);

public boolean promover() {
    return papel.compareAndSet(Papel.REPLICA, Papel.PRIMARIO);
    // Se duas threads tentarem simultaneamente, apenas uma retorna true
}

public boolean ehPrimario() {
    return papel.get() == Papel.PRIMARIO;
}
```

Fluxo de uma Leitura com Múltiplas Threads

```
Thread MQTT (callback Paho)
|
└─ submete para pool
Thread ingestao-N (ExecutorService)
|
├─ Sanitizador.valida(leitura) ← descarta outliers fisicamente impossíveis
├─ EstadoZona.registrarLeitura() ← escrita em ConcurrentHashMap +
ConcurrentLinkedDeque
├─ AvaliadorLimites.avaliar() ← compara com limite atual (leitura em
ConcurrentHashMap)
│   (se violação detectada)
│   └─ PublicadorAlertas.publicar() ← publica no MQTT
citypulse/alertas/{zona}
│   (se ehPrimario())
│   └─ ClienteReplicacao.enviar() ← envia atualização pelo stream gRPC
```

6.2 Concorrência nos Serviços TypeScript

Node.js usa um modelo single-threaded com event loop (libuv). Toda operação de I/O (MQTT, gRPC, HTTP, WebSocket, AMQP) é assíncrona e não bloqueia o event loop. A "concorrência" é cooperativa: múltiplas operações I/O podem estar em andamento simultaneamente, mas o código JavaScript em si é executado em sequência pela thread principal, eliminando condições de corrida em memória compartilhada.

Padrões de Assincronicidade por Serviço

Serviço	Padrão de Concorrência
Sensores Simulados	setTimeout recursivo com intervalo aleatório (5–15 s) para publicação não bloqueante. Sem estado compartilhado entre sensores no mesmo container de teste de carga.
Gateway de API	async/await em todos os handlers REST. RxJS internamente no NestJS. Múltiplas chamadas gRPC em Promise.all() quando necessário (SetThreshold Primário + Réplica em paralelo).
Dashboard (hooks)	useStatusPoll: setInterval a cada 4 s com fetch assíncrono — sem bloquear a renderização React. useAlertas: fetch inicial + listener de evento WebSocket em paralelo.
Worker de Manutenção	prefetch(1) no canal AMQP garante processamento sequencial de tarefas. Handlers implementados com async/await. Retry de conexão com setTimeout recursivo.

7. Serviços em Detalhe

7.1 Sensores Simulados

Linguagem: TypeScript | Runtime: Node.js 20 | Porta: 3001 (REST registro) | Arquivo principal: sensores-simulados/src/index.ts

Os sensores simulados reproduzem o comportamento de dispositivos IoT físicos. Cada instância do container representa um sensor individual, diferenciado por variáveis de ambiente (SENSOR_ID, ZONA_ID, TIPO). A classe abstrata BaseSensor gerencia o ciclo de vida: conexão ao broker MQTT, agendamento de publicações e serialização do payload JSON.

Comportamento de Publicação

Após a conexão MQTT ser estabelecida (evento connect), o sensor agenda a primeira publicação com setTimeout. Ao publicar, gera o próximo intervalo aleatório entre INTERVALO_MIN (5 s) e INTERVALO_MAX (15 s) e reagenda recursivamente. Esse padrão simula sensores assíncronos com jitter de tempo, evitando sincronismo artificial que mascararia problemas de concorrência no Coletor.

Simulação de Valores Realistas

Classe	Modelo de Geração	Outlier Intencional
TemperaturaSensor	caminhada aleatória $\pm 1,5^{\circ}\text{C}$ por leitura; range -5 a 45°C	5% chance de valor < -40 ou $> 60^{\circ}\text{C}$
QualidadeArSensor	caminhada ± 15 AQI + bias +0,45 (tendência de piora)	5% chance de < 0 ou > 1000 AQI
VagasEstacionamentoSensor	simulação de entrada/saída (-3 a $+3$ vagas); total 100 vagas	5% chance de valor < 0 ou > 100
ConsumoEnergiaSensor	fator multiplicador por hora: pico diurno (0.8–1.2) / vale noturno (0.2–0.3)	5% chance de valor negativo

O serviço de ingestão (ingestao.ts) é um servidor Express na porta 3001 com endpoint POST /sensores/registrar que mantém um Map em memória dos sensores ativos. Para testes de carga, o script load-test.ts instancia 64 sensores no mesmo processo (4 zonas $\times$ 4 tipos $\times$ 4 sensores por tipo) com intervalo acelerado de 1–3 s.

7.2 Coletor de Zona

Linguagem: Java 21 | Runtime: Eclipse Temurin 21 JRE | Portas: 50051–50054 (Primários), 50055–50058 (Réplicas), 60051–60054 (replicação interna) | Pacote: com.citypulse.coletor

O Coletor de Zona é o componente mais complexo do sistema. Implementado em Java 21 para demonstrar concorrência explícita com threads, pools de executores e estruturas de dados thread-safe, ele é responsável por receber leituras de sensores via MQTT, processá-las concorrentemente, detectar violações de limites, agregar métricas e manter o estado sincronizado com a Réplica.

Sequência de Inicialização (App.java)

1. Conecta ao broker MQTT com `setAutomaticReconnect=true`
2. Se `PAPEL_INICIAL=PRIMARIO`:
 - ↳ Cria `ClienteReplicacao` → abre stream gRPC bidirecional para a Réplica
3. Se `PAPEL_INICIAL=REPLICA`:
 - ↳ Sobe `ServidorReplicacao` (gRPC interno porta 60051)
 - ↳ Inicia watchdog de failover (`ScheduledExecutorService`, 1s período)
4. Sobe servidor gRPC `ZoneCollector` (porta 50051-50054)
5. Inicia `IngestaoMqtt`: subscreve `citypulse/sensores/{zona}/+ QoS 1`
 - ↳ Cria `ExecutorService` com 4 threads fixas (`ingestao-0` a `ingestao-3`)
6. Inicia Agregador: `ScheduledExecutorService` single-thread (1 min período)
7. Registra `ShutdownHook` para encerramento limpo (`SIGTERM` → `awaitTermination`)

Pacotes e Responsabilidades

Classe	Responsabilidade
<code>App.java</code>	Entry point e orquestrador de inicialização
<code>estado/EstadoZona.java</code>	Estado thread-safe: <code>ConcurrentHashMap</code> + <code>ConcurrentLinkedDeque</code> . Limite de 5.000 pts/tipo.
<code>grpc/ZoneCollectorServico.java</code>	Implementação gRPC: <code>GetZoneStatus</code> , <code>GetZoneHistory</code> , <code>SetThreshold</code>
<code>mqtt/IngestaoMqtt.java</code>	Pool de 4 threads consumindo mensagens MQTT e despachando para processamento
<code>mqtt/PublicadorAlertas.java</code>	Publica alertas no tópico MQTT <code>citypulse/alertas/{zona}</code> (só Primário)
<code>processamento/Sanitizador.java</code>	Filtra outliers fisicamente impossíveis antes de atualizar o estado
<code>processamento/AvaliadorLimites.java</code>	Compara leitura com limites configurados; dispara <code>PublicadorAlertas</code> se violado
<code>processamento/Agregador.java</code>	Calcula e armazena médias por minuto no histórico (background thread)
<code>replicacao/GerenciadorPapel.java</code>	<code>AtomicReference<Papel></code> com <code>compareAndSet</code> para promoção lock-free
<code>replicacao/ClienteReplicacao.java</code>	Lado Primário: abre stream gRPC + heartbeats 2 s + envio de deltas
<code>replicacao/ServidorReplicacao.java</code>	Lado Réplica: aplica atualizações de estado + watchdog failover 6 s

7.3 Gateway de API

Linguagem: TypeScript | Runtime: Node.js 20 | Framework: NestJS 10 | Porta: 3000 | Arquivo principal: `gateway-api/src/main.ts`

O Gateway é o ponto central de orquestração do sistema. Ele traduz requisições REST do Dashboard em chamadas gRPC para os Coletores, faz bridge entre MQTT (alertas) e WebSocket (push para o browser), e publica tarefas de manutenção na fila RabbitMQ. É o único componente que conhece os endereços de todos os Coletores.

Módulos NestJS

Módulo	Responsabilidade
<code>GrpcModule</code> / <code>GrpcService</code>	Carrega <code>citypulse.proto</code> em runtime (<code>@grpc/proto-loader</code>). Cria 2 clientes por zona

	(Primário + Réplica). Implementa callWithFailover() com redirect automático por código de erro gRPC.
ZonasModule / ZonasController	Expõe GET /api/zonas, GET /api/zonas/:id/status, GET /api/zonas/:id/historico, POST /api/zonas/:id/limites. Dispara tarefa RabbitMQ após SetThreshold bem-sucedido.
AlertasModule / AlertasService	Mantém ring buffer de 500 alertas em memória (array com limpeza ao chegar em 500). Expõe GET /api/alertas.
MqttModule / MqttService	Subscreve citypulse/alertas/+ no broker Mosquitto. Ao receber: adiciona ao AlertasService + emite para EventsGateway (WebSocket).
RabbitMqModule / RabbitMqService	Conecta ao RabbitMQ, declara fila durable. publishTask() publica tarefa com persistent:true.
WebsocketModule / EventsGateway	Servidor Socket.IO com rooms por zona. Clientes se inscrevem emitindo subscribe. emitAlerta() faz broadcast para o room da zona afetada.

Failover gRPC no Gateway

```

async callWithFailover(zona: string, method: string, req: any): Promise<any> {
  try {
    return await this.clients[zona].primary[method](req);    // tenta Primário
  } catch (err) {
    if (err.code === UNAVAILABLE || err.code === DEADLINE_EXCEEDED) {
      return await this.clients[zona].replica[method](req);  // redireciona para
    Réplica
    }
    throw err;
  }
}

```

7.4 Dashboard

Linguagem: TypeScript | Runtime: Node.js 20 | Framework: Next.js 14 / React 18 | Porta: 3002 | Diretório principal: dashboard/src/

O Dashboard é uma Single Page Application (SPA) construída com Next.js 14 App Router e React 18. Todas as páginas são marcadas como "use client" e exibem dados em tempo real combinando polling REST e eventos WebSocket.

Páginas e Funcionalidades

Página	Descrição
/ (Zonas)	Grid de ZonaCards com métricas atuais de cada zona; polling REST a cada 4 s via useStatusPoll hook
/alertas	Feed de alertas ao vivo; carga inicial via GET /api/alertas + updates em tempo real via Socket.IO
/replicacao	Exibe papel atual (PRIMARIO/REPLICA) de cada Coletor; indica visualmente o failover em tempo real
/historico	Gráfico de linha (LineChart.tsx) com histórico de DataPoints por zona e tipo de métrica
/limites	Formulário para configurar limites por zona e tipo; dispara POST /api/zonas/:id/limites

Hooks de Estado em Tempo Real

```
// useStatusPoll - polling REST
function useStatusPoll(zonaId: string, interval = 4000) {
  const [status, setStatus] = useState(null);
  useEffect(() => {
    const id = setInterval(async () => {
      const data = await fetchZoneStatus(zonaId);
      setStatus(data);
    }, interval);
    return () => clearInterval(id);
  }, [zonaId, interval]);
  return status;
}

// useAlertas - fetch inicial + WebSocket em tempo real
function useAlertas() {
  const [alertas, setAlertas] = useState([]);
  useEffect(() => {
    fetchAlertas().then(setAlertas); // carga inicial
    const socket = getSocket();
    socket.on('message', (alerta) =>
      setAlertas(prev => [alerta, ...prev].slice(0, 100)) // append + limit
    );
    return () => socket.off('message');
  }, []);
  return alertas;
}
```

7.5 Worker de Manutenção

Linguagem: TypeScript | Runtime: Node.js 20 | Biblioteca: amqplib | Arquivo principal: worker-manutencao/src/index.ts

O Worker é um consumer RabbitMQ que executa tarefas de manutenção assíncronas disparadas pelo Gateway após operações de configuração de limites. Opera com prefetch(1), garantindo que processa uma tarefa por vez e nunca sobrecarrega o sistema de arquivos com operações concorrentes.

Tipos de Tarefa

Tipo	Lógica
reconciliacao	Lê snapshots JSON do Primário e da Réplica em disco. Compara métricas campo a campo. Sobrescreve snapshot da Réplica com dados do Primário para corrigir divergências.
downsampling	Lê snapshot do Primário. Identifica DataPoints com mais de DOWNSAMPLING_IDADE_MIN minutos (default 60). Agrega por tipo+hora (média). Substitui pontos antigos por agregados — reduz volume histórico.
relatorio	Lê snapshot do Primário. Calcula min/max/média por tipo de métrica. Grava JSON em relatorios/{zona_id}-{timestamp}.json para análise posterior.

O armazenamento usa o sistema de arquivos local com `fs.readFileSync` e `fs.writeFileSync` (operações síncronas). Isso é seguro porque o `prefetch(1)` garante que apenas uma tarefa é processada por vez, eliminando condições de corrida no acesso aos arquivos.

8. Armazenamento de Dados

O CityPulse não utiliza banco de dados relacional ou NoSQL. O armazenamento é distribuído entre memória (para dados quentes e tempo real) e sistema de arquivos (para snapshots e relatórios de manutenção). Essa escolha é adequada ao escopo acadêmico do projeto e elimina dependências de infraestrutura de banco de dados.

Componente	Mecanismo	Dados Armazenados	Características
Coletor de Zona (Java)	ConcurrentHashMap + ConcurrentLinkedDeque na JVM	Métricas atuais por tipo, histórico de até 5.000 pontos por tipo, limites configurados	Estado perdido ao reiniciar o container (design intencional)
Gateway de API (NestJS)	Array JavaScript em memória (AlertasService)	Últimos 500 alertas recebidos via MQTT (ring buffer simples)	Alerta mais antigo descartado ao atingir 500
Worker de Manutenção	Sistema de arquivos local (JSON)	Snapshots em dados/{zona}-{papel}.json; Relatórios em relatorios/{zona}-{ts}.json	Durável entre reinicializações; operações síncronas (prefetch=1 garante segurança)

Estrutura do Snapshot JSON (Worker)

```
{
  "zona_id": "centro",
  "papel": "PRIMARIO",
  "timestamp": "2026-06-28T14:30:00Z",
  "metricas": {
    "temperatura": { "valor": 28.5, "unidade": "°C", "timestamp": "2026-06-28T14:29:55Z" },
    "qualidade_ar": { "valor": 142, "unidade": "AQI", "timestamp": "2026-06-28T14:29:58Z" }
  },
  "historico": {
    "temperatura": [
      { "valor": 28.5, "timestamp": "2026-06-28T14:29:55Z" },
      { "valor": 28.3, "timestamp": "2026-06-28T14:29:40Z" }
    ]
  }
}
```

9. Separação de Instâncias e Deploy

9.1 Ambiente de Desenvolvimento (Docker Compose)

Em desenvolvimento, todos os 30+ containers rodam na mesma máquina orquestrados pelo docker-compose.yml na raiz do projeto. Todos os containers compartilham a rede bridge nomeada citypulse, comunicando-se pelo hostname do serviço como endereço DNS (ex.: mosquito:1883, coletor-centro:50051).

Containers no Docker Compose

Qtd	Nome	Imagem Base	Portas Host	Função
1	mosquitto	Eclipse Mosquitto	1883, 9001	Broker MQTT
1	rabbitmq	RabbitMQ 3.13	5672, 15672	Broker AMQP + UI
1	ingestao-sensores	Node.js 20	3001	Registro de sensores REST
16	sensor-{zona}-{tipo}	Node.js 20	—	4 zonas × 4 tipos de sensor
4	coletor-{zona}	JRE 21	50051-50054	Coletores Primários
4	coletor-{zona}-replica	JRE 21	50055-50058, 60051-60054	Coletores Réplicas
1	gateway	Node.js 20 (NestJS)	3000	Gateway de API
1	dashboard	Node.js 20 (Next.js)	3002	Dashboard Web
1	worker-manutencao	Node.js 20	—	Consumer AMQP

9.2 Ambiente de Produção (AWS EC2)

Em produção, o sistema é distribuído em 5 instâncias EC2 separadas (Amazon Linux 2023) em uma VPC privada, simulando um ambiente real de data center com isolamento de rede. Cada instância executa um subconjunto de containers com seu próprio docker-compose.yml.

Instância	Tipo EC2	Serviços	Justificativa
A — Infraestrutura	t3.micro	Mosquitto + RabbitMQ	Brokers MQTT e AMQP acessíveis internamente por todas as instâncias
B — Primários	t3.small	4 Coletores Java Primários (centro/norte/sul/leste)	Processamento de leituras e publicação de alertas — mais exigente em CPU/memória
C — Réplicas	t3.micro	4 Coletores Java Réplicas	Stand-by para failover; recebe stream de replicação dos Primários da Instância B
D — Backend	t3.micro	Gateway NestJS + Worker de Manutenção	API REST/WebSocket e processamento de tarefas AMQP
E — Frontend/Sensores	t3.micro	Dashboard Next.js + Script de carga de sensores	Interface web pública e geração de carga de teste

Configuração de Rede AWS

- VPC: Amazon VPC padrão com Security Group citypulse-sg

- Regra inbound self-referencing: permite tráfego entre todas as instâncias do mesmo Security Group
- Portas expostas para a internet: 3000 (Gateway REST/WS) e 3002 (Dashboard)
- Portas internas (apenas VPC): 1883 MQTT, 5672 AMQP, 50051–50058 gRPC, 60051–60054 replicação
- Custo estimado: US\$ 3–5 para 3–4 dias de execução completa

Dockerfiles — Estratégias de Build

Serviço	Base	Estratégia	Detalhe
Sensores / Gateway / Worker	Node.js 20 Alpine	Imagem única	npm ci → ts-node direto (sem transpile para produção)
Dashboard	Node.js 20 Alpine	Imagem única	next build → next start na porta 3002
Coletor de Zona	Maven 3.9 + JRE 21	Multistage	Stage 1: mvn package -DskipTests → fat JAR. Stage 2: JRE 21 mínimo → copia JAR

10. Fluxos de Dados Principais

Fluxo 1 — Leitura Normal de Sensor

1. Sensor-Centro-Temperatura gera valor (ex.: 29.2 °C) com setTimeout aleatório
2. Publica JSON no tópico MQTT citypulse/sensores/centro/temperatura (QoS 1)
3. Mosquitto entrega ao Coletor-Centro (inscrito no tópico centro/+)
4. IngestaoMqtt.java recebe no callback e submete ao pool de 4 threads
5. Thread ingestao-2 processa:
 - a. Sanitizador.valida() → valor dentro dos limites físicos → aceito
 - b. EstadoZona.registrarLeitura() → ConcurrentHashMap + ConcurrentLinkedDeque
 - c. AvaliadorLimites.avaliar() → valor abaixo do limite → sem alerta
6. ClienteReplicacao envia atualização pelo stream gRPC para Coletor-Centro-Réplica
7. ServidorReplicacao.java aplica atualização em seu próprio EstadoZona
8. Dashboard (polling a cada 4 s) faz GET /api/zonas/centro/status
9. Gateway chama GetZoneStatus gRPC no Coletor-Centro
10. Resposta percorre: Coletor → Gateway → Dashboard → React re-render

Fluxo 2 — Detecção e Propagação de Alerta

1. Sensor-Sul-QualidadeAr publica AQI = 280 (acima do limite configurado de 200)
2. Thread ingestao-N no Coletor-Sul executa:
 - a. Sanitizador → aceito (280 é fisicamente plausível)
 - b. EstadoZona.registrarLeitura() → salva leitura
 - c. AvaliadorLimites.avaliar() → 280 > 200 → VIOLA LIMITE
 - d. Coletor verifica: ehPrimario() == true → PublicadorAlertas.publicar()
3. Alerta publicado em MQTT: citypulse/alertas/sul (QoS 1)
4. Gateway (MqttService) recebe o alerta:
 - a. AlertasService.add(alerta) → adiciona ao ring buffer de 500
 - b. EventsGateway.emitAlerta(alerta) → broadcast para room zona:sul (Socket.IO)
5. Dashboard (useAlertas hook) recebe evento WebSocket → atualiza estado React
6. Alerta aparece em tempo real na página /alertas do Dashboard

Fluxo 3 — Failover Automático de Réplica

1. Coletor-Norte-Primário cai (container parado, falha de rede ou OOM)
2. Stream gRPC de replicação é interrompido com erro
3. Cenário A — erro no stream:
ServidorReplicacao.onError() detecta imediatamente
↳ chama gerenciador.promover() → compareAndSet(REPLICA → PRIMARIO) retorna true
4. Cenário B — timeout do watchdog (6 s sem heartbeat):
Thread replicacao-watchdog detecta timestamp.now() - ultimoHeartbeat > 6000 ms
↳ chama gerenciador.promover() → idêntico
5. Ambos os cenários são idempotentes: se os dois ocorrerem, compareAndSet garante que apenas a primeira chamada promove (retorna true), a segunda não-op

6. Coletor-Norte-Réplica (agora PRIMÁRIO):
 - a. `IngestaoMqtt.messageArrived()` verifica `ehPrimario()` → true
 - b. Passa a processar leituras MQTT e publicar alertas
 - c. gRPC `GetZoneStatus` retorna `papel_no_momento = "PRIMARIO"`
7. Gateway detecta falha na próxima chamada ao Primário original:
↳ `callWithFailover()` → código UNAVAILABLE → redireciona para Réplica (novo Primário)
8. Dashboard `/replicacao` mostra badge PRIMARIO na Réplica após o próximo poll (4 s)

Fluxo 4 — Configuração de Limite com Consistência Forte

1. Usuário acessa `/limites` no Dashboard, define limite de temperatura = 35 °C para Centro
2. Dashboard faz POST `/api/zonas/centro/limites` { tipo: "temperatura", maximo: 35 }
3. Gateway recebe requisição REST no `ZonasController`
4. Gateway chama `SetThreshold` gRPC no Coletor-Centro-Primário
 - a. Primário atualiza seu `EstadoZona.limites`
 - b. Primário chama `aplicarLimite` gRPC na Réplica (`ServidorReplicacao`)
 - c. Réplica atualiza seu `EstadoZona.limites`
 - d. Réplica retorna `sucesso=true` para o Primário
 - e. Primário retorna `sucesso=true` para o Gateway
5. Gateway publica tarefa AMQP { tipo: "reconciliacao", zona_id: "centro" }
6. Worker de Manutenção recebe a tarefa (prefetch=1):
 - a. Lê snapshots JSON do Primário e Réplica
 - b. Verifica consistência dos dados
 - c. Sobrescreve snapshot da Réplica se necessário
7. Gateway responde 200 OK ao Dashboard
8. `AvaliadorLimites` nos próximos processos de leitura usa o novo limite de 35 °C

11. Considerações Finais

Requisitos da Disciplina Atendidos

Requisito	Status	Evidência no Sistema
Múltiplos componentes distribuídos	✓	5 tipos de serviço, 30 containers, 5 VMs em produção
Múltiplas linguagens de programação	✓	Java 21 (Coletor) + TypeScript (todos os demais)
Interação síncrona bloqueante	✓	REST HTTP e gRPC (incluindo SetThreshold com await duplo)
Interação assíncrona	✓	MQTT pub/sub, RabbitMQ, WebSocket
Concorrência explícita com threads	✓	Pool de 4 threads, ScheduledExecutorService, AtomicReference
Acesso concorrente a dados compartilhados	✓	EstadoZona: gRPC + 4 threads ingestão + Agregador simultâneos
Particionamento de dados	✓	4 zonas = 4 partições com Coletores independentes
Replicação	✓	Primário-Réplica por zona via stream gRPC com heartbeat 2 s
Failover automático	✓	Promoção lock-free da Réplica após 6 s sem heartbeat
Consistência forte em escrita	✓	SetThreshold exige confirmação síncrona de Primário + Réplica
Deploy distribuído real	✓	5 EC2 distintas na AWS com comunicação via VPC privada

Escolhas de Design Notáveis

- **Java no coração crítico:** Java para o Coletor:

Java 21 foi escolhido para o componente que exige concorrência explícita justamente por oferecer um ecossistema maduro de programação concorrente (java.util.concurrent, estruturas lock-free, JVM com GC otimizado) que permite demonstrar os conceitos da disciplina de forma clara e controlada.

- **Protocol Buffers como contrato:**

O uso de um único arquivo .proto compartilhado (proto/citypulse.proto) como interface entre Java e TypeScript garante tipo forte em ambas as linguagens e detecta incompatibilidades de contrato em tempo de compilação, não em tempo de execução.

- **Quatro protocolos com papéis distintos:**

MQTT para I/O assíncrono de alta frequência (sensores); gRPC para leitura consistente de estado e replicação com tipos fortes; WebSocket para push bidirecional sem polling; AMQP para tarefas de background com durabilidade. Cada protocolo é usado onde suas características são vantajosas.

- **Ring buffer para alertas:**

O AlertasService no Gateway mantém no máximo 500 alertas em memória, descartando os mais antigos. Essa escolha simples elimina a necessidade de banco de dados para o feed de alertas e garante bounded memory usage independente da frequência de violações de limites.

Limitações e Extensões Possíveis

- Persistência: o estado dos Coletores é perdido ao reiniciar. Uma extensão natural seria persistir snapshots periodicamente no Worker e restaurar na inicialização do Coletor.
- Reconexão do Primário: após falha e recuperação, o Primário original sobe como Réplica. O sistema não implementa re-eleição automática — um operador deve reiniciar o Primário original como Réplica manualmente.
- Autenticação: CORS está aberto e não há autenticação nos endpoints REST ou WebSocket — adequado para demonstração acadêmica, mas insuficiente para produção real.
- Observabilidade: não há coleta de métricas (Prometheus/Grafana) ou rastreamento distribuído (OpenTelemetry). Logs são escritos em stdout dos containers.
- Escala horizontal: o particionamento por zona é fixo em 4. Para escalar dinamicamente, seria necessário service discovery (Consul/etcd) e reparticionamento dinâmico.